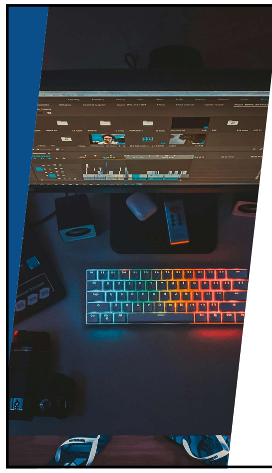
Python Programming Foundations

Day 1 - Coding/Course Setup

"Creating the working environment"

1

Setup Is Part Of The Course



Before writing Python comfortably, your computer needs a stable setup.

The goal is not to memorize setup steps. The goal is to create a reliable place to run your code.

2

Setup Path

Course setup path:

- Python ->
- project folder ->
- `.venv` ->
- VS Code ->
- GitHub

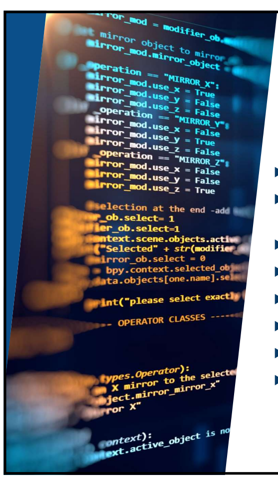
Each piece supports a different part of the workflow.

3

What Success Look Like

- check your Python version
- open a project in VS Code
- select a `.venv` interpreter
- run a small `.py` file
- see your code in GitHub or prepare it for submission

4



What We Will Use Today

- Python from `python.org`
- PowerShell or terminal verification commands
- project folders
- `.venv` virtual environments
- Visual Studio Code
- Microsoft Python extension
- VS Code interpreter selection
- VS Code source control / GitHub integration

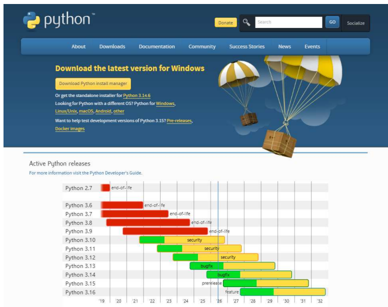
5

What We Will "Skip" For Now

- advanced dependency management
- publishing packages
- complex Git branching
- pull request reviews
- deployment
- Python pre-release versions

6

Download Python



url: python.org/downloads

7

Windows – Two options

Either option will work
(The top one is the easiest)

8

New: Python Install Manager

(Your textbook does not mention this)

9

Install Python

- The Install manager will make sure that a usable `py` or `python` command is created.
- If the installer offers PATH-related options, you should enable the option that lets Python run from the terminal.
- Once the process is finished:
  - Open a terminal (powershell)
  - On the command line type `py`
  - Optionally, next type the command `help'

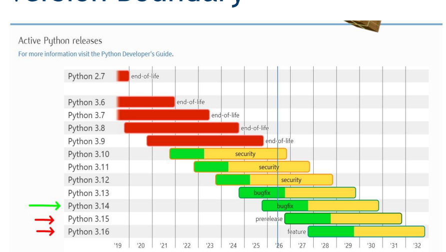
10

Verify Python

- Once the process is finished:
  - Open a Terminal or Powershell and run one of the following commands:
    - `python --version`
    - `py --version`
    - `python -m pip --version`
  - Optionally, launch the interactive python shell and try the help command
    - `py`
    - Then type: `help`
    - Within help: `keywords`, `modules`, `symbols` or `topics`

11

Version Boundary



- Use a ***stable*** Python release.
- Do not use pre-release versions for this course unless the instructor tells you to.
- *"You don't want to get 'bit'!"* 😝

12

Create A Course Folder

Create a folder, or folder structure for your course work.

Keep your projects and assignments organized from the start.

13

What A Virtual Environment Does

- A virtual environment is a local Python workspace for one project.
- It helps keep project packages separate from the rest of your computer.
- It also allows for different “versions” of python to “co-exist” on one machine. This will be important when working on legacy software applications.

14

Create `.venv`

From the project folder, using the terminal or powershell, create a virtual environment:

- `py -m venv .venv`

If `py` is not available, use:

- `python -m venv .venv`

15

Activate `.venv` In PowerShell

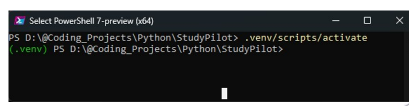
Windows PowerShell activation:

Command:

- `.\.venv\Scripts\Activate.ps1`

When active, the terminal prompt usually shows:

- (.venv)



16

Why `.venv` Matters Later

Later: when projects use packages, `.venv` helps keep installs project-specific.

For Week 1: the goal is simply to know what `.venv` is and how to create/select it.

17

Install Visual Studio Code

18

Difference between VS Code and Visual Studio

Visual Studio is a true IDE - Integrated Developer Environment.

- Heavy footprint (takes a lot of memory)
- Great for C# and .NET framework development
- Doesn't play well on an iOS machines

VS Code is a code editor, not a true IDE ... but can function as a full-IDE by installing the extensions

- Widely used!!
- Smaller footprint (faster load and execution)
- Updated very frequently
- Supports multiple development languages and frameworks

19

Download VS Code

url: code.visualstudio.com/download

20

VS Code Extensions

21

Install Python Extensions

Recommended VS Code extensions:

- Python
- Pylance
- Python Environments
- Python Extension Pack, optional (Don Jayamanne)
- Python Development Extensions Pack, optional
- Jupyter, optional

Use trusted publishers such as Microsoft.

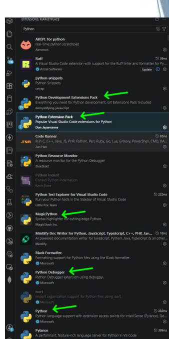
22

Install Python Extensions

Select the Extensions button on the left side bar.

Search for the desired extensions

Click on the "Install"



23

Install Optional Additional Extensions

Some cool extensions

- Better Comments
- Better Todo Tree
- Bracket Pair Color DLW
- Colorful Dark+
- Colorful Comments
- #region folding for VS Code
- Powershell Extension Pack
- Print

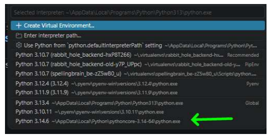
- Git Extensions Pack
- Git Graph

24

Select The Python Interpreter

In VS Code:

1. Open the project folder.
2. Open the Command Palette (Ctrl+Shift+P).
3. Choose `Python: Select Interpreter`.
4. Select the interpreter inside `.venv`.



25

VS Code And `.venv`

VS Code can detect and use virtual environments.

When the `.venv` interpreter is selected, new VS Code terminals can activate that environment automatically.

If it is not selected, you will need to manually activate the virtual environment using the activate command

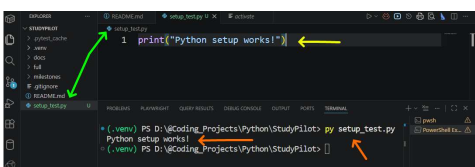
26

Run A Test File

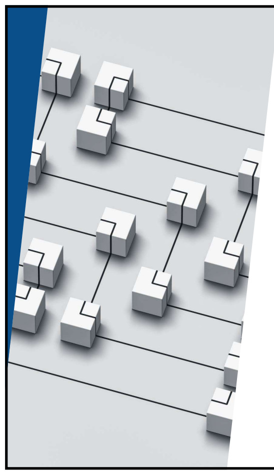
Create a file: `setup_test.py`

Add the line: `print("Python setup works!")

Execute the file from integrated terminal: `py setup_test.py`



27



GitHub Submission Paths

There are two common GitHub paths:

- Git CLI commands *(you should learn these regardless)*
- VS Code Source Control and GitHub extensions

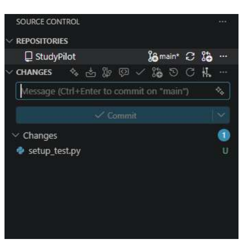
Both paths should preserve code changes clearly.

28

Source Control In VS Code

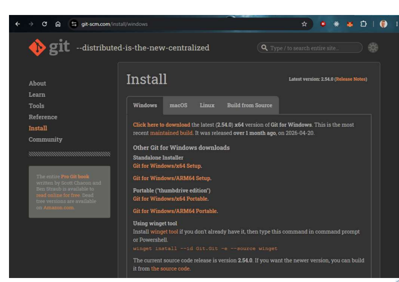
VS Code has a Source Control view for Git work.

You can see changed files, write a commit message, commit, and sync without typing every Git command manually.



29

Git Install for Windows



url: git-scm.com/install/windows

30

Initialize git for your project(s)

Using either VS Code or the CLI – initialize git (locally) for your project.

- `git init`

Note:

- This does not setup a GitHub repository in the cloud. It only sets it up local.
- Later we will connect the local with the GitHub repo

31

GitHub Extensions

Useful GitHub-related VS Code extensions:

- GitHub Pull Requests
- GitHub Repositories

These can help with GitHub sign-in, repositories, commits, and pull-request workflows.

32

GitHub Vocabulary

Core GitHub Words

Keyword	Definition/Use
Repository	Project storage
Clone	Copy repository to your computer
Commit	Save a meaningful change
Push	Send commits to GitHub
Sync	Pull and push changes through VS Code

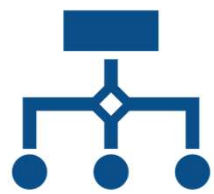
33

CLI Or VS Code

Git CLI and VS Code can both support the same workflow:

change files ->
review changes ->
commit ->
push or sync

Use either path (or the one your instructor approves) for the assignment.



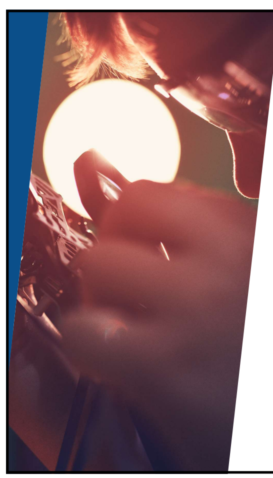
34

Setup Verification Checklist

Setup checklist:

- Python version command works
- project folder exists
- `.venv` exists
- VS Code opens the folder
- VS Code selects the `.venv` interpreter
- `setup_test.py` runs
- GitHub workflow is understood or ready to practice

35



Troubleshooting Layers

When setup fails, locate the layer:

- Python install
- terminal command
- project folder
- `.venv`
- VS Code interpreter
- Git or GitHub sign-in

Fix one layer at a time.

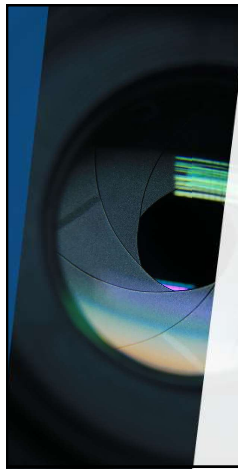
36

Ready For First Programs

You are ready for Week 1 coding when:

- Python runs
- VS Code opens your project
- `.venv` is available
- a tiny `.py` file prints output
- your submission path is clear

37



Demo/ Walkthrough

38

Demo / Walkthrough Notes (Review)

1. Open `python.org/downloads/`.
2. Show the latest stable Python release.
3. Install or verify Python.
4. Run version checks in PowerShell.
5. Create a project folder.
6. Create `.venv`.
7. Open VS Code.
8. Install Python-related extensions.
9. Select the `.venv` interpreter.
10. Run `setup_test.py`.
11. Source Control and GitHub extension locations in VS Code.

39

Your Turn

Install Python

Run version checks

Install VS Code if needed

Setup Project Folders

Create .venv

Open VS Code

Install Extensions

Select the interpreter

Run `setup_test.py`

Initialize git locally

Commit your change(s)

40

Setup Evidence

Recommended evidence:

- screenshot or copied terminal text showing Python version
- screenshot or note showing `.venv` exists
- screenshot or copied output from `setup_test.py`
- confirmation that VS Code selected the `.venv` interpreter
- confirmation of local git, GitHub sign-in, or instructor-approved submission path

Keep this as setup evidence only or for troubleshooting.

This is NOT graded.

41

Great!

Now let's start the class!

42